

Bootstrap iterations for infit MSQ: FWER p-values vs simulation cutoffs

Extending the 100/200/400 comparison in Johansson (2025) to FWER-adjusted p-values

Magnus Johansson

2026-07-27

Table of contents

Purpose	1
Why the two rules should behave differently	1
Data generation	2
Study A: error rate and detection as a function of B	2
Family-wise error rate under a true Rasch model ($k = 0$)	3
Detection, one misfitting on-target item ($k = 1$)	5
Detection and false positives, three misfitting items ($k = 3$)	5
Figure	6
Study B: Monte-Carlo stability of the decision	7
Decision instability	8
Spread of the adjusted p-value across seeds	8
Study C: the small-sample trade, and buying detection honestly	9
Detection and false positives at $n = 150$ and 250 ($k = 3$)	11
Family-wise false positive rate under a true model ($k = 0$)	11
Direction of the false flags ($k = 3$, fitting items only)	11
Expected flags per analysis ($k = 3$)	12
Conclusions	13
Practical recommendations	13
Note on other functions using this helper	14
References	14

Purpose

Johansson (2025), Study 1, compared 100, 200 and 400 bootstrap iterations for determining simulation-based **cutoffs** for conditional infit MSQ, and found that *more* iterations gave *lower* detection rates (80.0 / 76.8 / 69.4 percent at $n = 150$, on-target misfitting item), with false positives appearing only at $n \geq 500$. The recommendation was 100 iterations for $n \leq 250$ and 200-400 for larger samples.

easyRasch2 has since added an alternative decision rule for the same bootstrap: Monte-Carlo **p-values** with multiplicity correction, via `RMItemInfit(p_value = TRUE, correction = "fwer")` — the Westfall-Young studentised-max step-down. Its documentation currently states that at least 1000 iterations are recommended, that the FWER correction is “liberal when the simulation is small”, and that marginal p-values are “well calibrated even at a few hundred iterations”.

That guidance has not been established by an iteration sweep. `dev/infit_dgp_fwer.R` fixed $B = 1000$ and varied the data-generating process instead. This document sweeps B directly.

Why the two rules should behave differently

The mechanisms are not the same, and this predicts opposite directions:

- **HDCI cutoffs** need the *extreme quantiles* of the null distribution (`hdci_width = 0.999` implies the .0005 and .9995 quantiles). With $B = 100$ draws those quantiles cannot be estimated — they are effectively the observed min and max — so the interval is too narrow and the rule over-flags. More iterations widen the interval, which *reduces* detection. That is the artifact visible in Johansson (2025), not a benefit of few iterations.
- **Monte-Carlo p-values** use the form $(1 + \text{count}) / (B + 1)$, which is exactly valid for any B under exchangeability. Increasing B should therefore *not* change the error rate, only reduce the variance of the estimate. The floor is $1 / (B + 1)$: .0099, .0050, .0025, .0010 for $B = 100, 200, 400, 1000$ — all below $\alpha = .05$, so no hard floor problem at conventional α .
- **The FWER step-down** is the exception. It studentises by the bootstrap mean and SD, both estimated from the same B draws, and then takes a maximum over items. Noise in the studentisation inflates the simulated maxima inconsistently, which is the stated reason for expecting liberal behaviour at small B .

Two things are therefore measured: the **error rate** as a function of B (Study A), and the **Monte-Carlo stability of the decision** for a fixed dataset (Study B), which the error rate alone hides.

```

suppressMessages({
  library(easyRasch2)
  library(eRm)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

BS      <- c(100, 200, 400, 1000)
NS      <- c(150, 250, 500, 1000)
NMISFITS <- c(0, 1, 3)
R_REPS  <- 100
NCORES  <- 12          # performance cores only (M4 Max)
ALPHA   <- 0.05

```

Data generation

Identical to `sx2_itemfit_sim.qmd`, which follows Johansson (2025): 20 dichotomous items, locations `runif(20, -2, 2)`, person locations bivariate normal with SD 1.5 and inter-dimensional correlation 0.15, misfit induced by loading items on the second dimension, misfitting items at approximately 0, -1 and -2 logits.

```

make_master <- function(n_misfit, N = 10000, seed = 20250727) {
  set.seed(seed)
  locs <- runif(20, -2, 2)
  misfit_idx <- integer(0)
  for (tg in c(0, -1, -2)) {
    cand <- setdiff(order(abs(locs - tg)), misfit_idx)[1]
    misfit_idx <- c(misfit_idx, cand)
  }
  use <- misfit_idx[seq_len(n_misfit)]
  W <- cbind(rep(1, 20), rep(0, 20))
  if (n_misfit > 0) W[use, ] <- matrix(c(0, 1), nrow = length(use), ncol = 2,
                                       byrow = TRUE)
  Sigma <- matrix(c(1.5^2, .15 * 1.5^2, .15 * 1.5^2, 1.5^2), 2, 2)
  set.seed(seed + n_misfit)
  th <- MASS::mvrnorm(N, mu = c(0, 0), Sigma = Sigma)
  d <- as.data.frame(eRm::sim.xdim(persons = th, items = locs,
                                   Sigma = Sigma, weightmat = W))
  names(d) <- paste0("V", 1:20)
  list(data = d, locs = locs, misfit = use)
}

masters <- lapply(NMISFITS, make_master)
names(masters) <- as.character(NMISFITS)
misfit_all <- masters[["3"]]$misfit

```

Study A: error rate and detection as a function of B

All four decision rules are read off the **same** bootstrap run, so differences are attributable to the rule and not to simulation noise between rules.

```

RES_A <- "infit_fwer_iterations_A.rds"

one_rep_A <- function(rep, k, n, B) {
  set.seed(1e7 * k + 1e4 * n + 1e2 * rep + log10(B))
  md <- masters[[as.character(k)]]$data
  d <- md[sample(nrow(md), n), ]
  out <- data.frame(item = 1:20, hpci = NA_integer_, fwer = NA_integer_,
                    bh = NA_integer_, uncorr = NA_integer_,
                    padj_fwer = NA_real_, p_marg = NA_real_)
  try({

```

```

co <- RMitemInfitCutoff(d, iterations = B, parallel = FALSE,
                        seed = rep, verbose = FALSE)
a <- RMitemInfit(d, cutoff = co, output = "dataframe")
out$hdci <- as.integer(a$Flagged[match(names(d), a$Item)] != "")
f <- suppressWarnings(
  RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
              alpha = ALPHA, output = "dataframe"))
i <- match(names(d), f$Item)
out$fwer <- as.integer(f$padj_infit[i] < ALPHA)
out$padj_fwer <- f$padj_infit[i]
out$p_marg <- f$p_infit[i]
out$uncorr <- as.integer(f$p_infit[i] < ALPHA)
bh <- suppressWarnings(
  RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
              alpha = ALPHA, output = "dataframe"))
out$bh <- as.integer(bh$padj_infit[match(names(d), bh$Item)] < ALPHA)
}, silent = TRUE)
out
}

gridA <- expand.grid(rep = 1:R_REPS, B = BS, n = NS, k = NMISFITS)

if (file.exists(RES_A)) {
  simA <- readRDS(RES_A)
} else {
  t0 <- Sys.time()
  resA <- mclapply(seq_len(nrow(gridA)),
                  function(i) one_rep_A(gridA$rep[i], gridA$k[i],
                                         gridA$n[i], gridA$B[i]),
                  mc.cores = NCORES)
  simA <- list(res = resA, grid = gridA, misfit = misfit_all,
              elapsed_min = as.numeric(Sys.time() - t0, "mins"))
  saveRDS(simA, RES_A)
}

longA <- do.call(rbind, lapply(seq_len(nrow(simA$grid)),
                              function(i) cbind(simA$grid[i, ], simA$res[[i]]))) |>
  mutate(is_misfit = mapapply(function(k, it) it %in% simA$misfit[seq_len(k)],
                              k, item))
cat("elapsed:", round(simA$elapsed_min, 1), "min | failed:",
    sum(is.na(longA$hdci)), "of", nrow(longA), "\n")

```

```
elapsed: 63.2 min | failed: 0 of 96000
```

Family-wise error rate under a true Rasch model (k = 0)

The quantity the FWER correction is meant to control is $P(\text{any item flagged})$ per dataset, which should be at or below $\alpha = .05$.

```

longA |>
  filter(k == 0) |>
  pivot_longer(c(hdci, fwer, bh, uncorr), names_to = "rule", values_to = "flag") |>
  group_by(rule, B, n, rep) |>
  summarise(any_flag = as.integer(sum(flag, na.rm = TRUE) > 0), .groups = "drop") |>
  group_by(rule, B, n) |>
  summarise(v = 100 * mean(any_flag), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rule, B)

```

rule	B	n=150	n=250	n=500	n=1000
bh	100	0	0	0	0

rule	B	n=150	n=250	n=500	n=1000
bh	200	1	0	1	0
bh	400	4	2	3	9
bh	1000	1	4	2	4
fwer	100	9	3	2	10
fwer	200	5	2	3	9
fwer	400	5	2	2	8
fwer	1000	2	2	3	6
hdci	100	38	23	33	28
hdci	200	19	15	14	14
hdci	400	9	4	8	13
hdci	1000	2	3	3	5
uncorr	100	60	59	63	67
uncorr	200	58	60	63	69
uncorr	400	61	56	63	69
uncorr	1000	59	53	61	68

Per-item false positive rate over the same datasets:

```
longA |>
  filter(k == 0) |>
  pivot_longer(c(hdci, fwer, bh, uncorr), names_to = "rule", values_to = "flag") |>
  group_by(rule, B, n) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 2))) |>
  arrange(rule, B)
```

rule	B	n=150	n=250	n=500	n=1000
bh	100	0.00	0.00	0.00	0.00
bh	200	0.10	0.00	0.10	0.00
bh	400	0.30	0.15	0.15	0.45
bh	1000	0.05	0.20	0.10	0.30
fwer	100	0.50	0.15	0.10	0.50
fwer	200	0.35	0.10	0.15	0.45
fwer	400	0.30	0.10	0.10	0.40
fwer	1000	0.10	0.10	0.15	0.35
hdci	100	2.30	1.35	1.95	1.45
hdci	200	1.10	0.75	0.80	0.70
hdci	400	0.50	0.20	0.40	0.65
hdci	1000	0.10	0.15	0.15	0.25
uncorr	100	4.60	4.25	5.10	5.05
uncorr	200	4.55	4.20	4.95	5.15
uncorr	400	4.85	3.90	4.80	4.90

rule	B	n=150	n=250	n=500	n=1000
uncorr	1000	4.25	4.55	4.60	4.70

Detection, one misfitting on-target item (k = 1)

Directly comparable to Figure 2 in Johansson (2025).

```
longA |>
  filter(k == 1, is_misfit) |>
  pivot_longer(c(hdci, fwer, bh, uncorr), names_to = "rule", values_to = "flag") |>
  group_by(rule, B, n) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rule, B)
```

rule	B	n=150	n=250	n=500	n=1000
bh	100	1	0	0	10
bh	200	8	13	23	54
bh	400	99	100	100	100
bh	1000	100	100	100	100
fwer	100	99	100	100	100
fwer	200	99	100	100	100
fwer	400	99	100	100	100
fwer	1000	100	100	100	100
hdci	100	100	100	100	100
hdci	200	100	100	100	100
hdci	400	99	100	100	100
hdci	1000	99	100	100	100
uncorr	100	100	100	100	100
uncorr	200	100	100	100	100
uncorr	400	100	100	100	100
uncorr	1000	100	100	100	100

Detection and false positives, three misfitting items (k = 3)

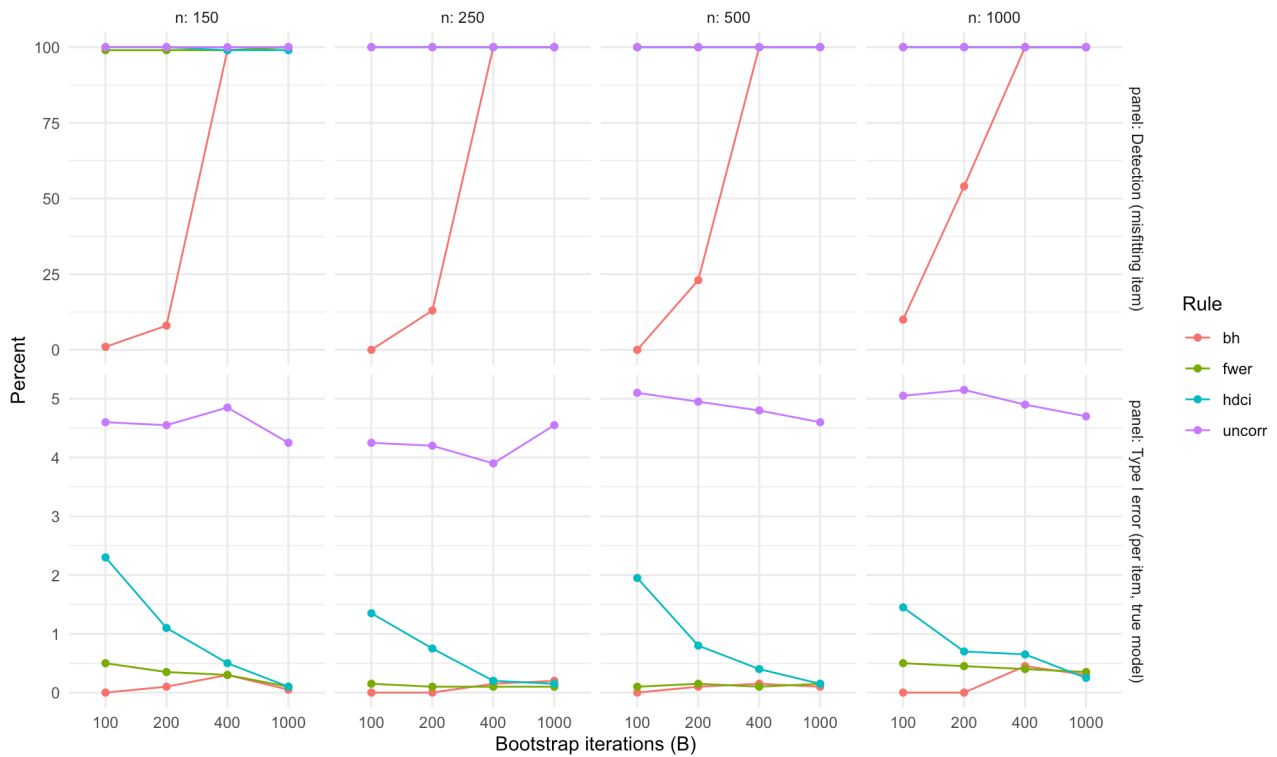
```
longA |>
  filter(k == 3) |>
  pivot_longer(c(hdci, fwer, bh, uncorr), names_to = "rule", values_to = "flag") |>
  mutate(type = ifelse(is_misfit, "detection", "false_pos")) |>
  group_by(rule, B, n, type) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(type, rule, B)
```

rule	B	type	n=150	n=250	n=500	n=1000
bh	100	detection	13.0	34.0	83.0	99.0
bh	200	detection	69.7	92.3	100.0	100.0
bh	400	detection	76.3	94.0	100.0	100.0

rule	B	type	n=150	n=250	n=500	n=1000
bh	1000	detection	79.3	91.7	100.0	100.0
fwer	100	detection	73.0	92.3	100.0	100.0
fwer	200	detection	71.7	91.0	99.3	100.0
fwer	400	detection	70.7	90.7	99.7	100.0
fwer	1000	detection	75.7	90.3	99.7	100.0
hdci	100	detection	81.3	94.3	100.0	100.0
hdci	200	detection	75.3	91.3	99.7	100.0
hdci	400	detection	68.0	88.3	99.3	100.0
hdci	1000	detection	62.7	85.3	99.7	100.0
uncorr	100	detection	90.3	98.7	100.0	100.0
uncorr	200	detection	90.3	99.0	100.0	100.0
uncorr	400	detection	91.3	99.0	100.0	100.0
uncorr	1000	detection	91.0	97.7	100.0	100.0
bh	100	false_pos	0.9	2.4	9.9	33.1
bh	200	false_pos	1.1	2.6	10.4	38.6
bh	400	false_pos	1.4	3.2	12.6	39.9
bh	1000	false_pos	1.2	3.2	13.5	38.6
fwer	100	false_pos	0.6	1.1	5.5	16.7
fwer	200	false_pos	0.4	0.7	5.2	15.5
fwer	400	false_pos	0.5	0.7	4.8	15.7
fwer	1000	false_pos	0.4	0.8	4.4	14.7
hdci	100	false_pos	3.6	6.2	14.5	33.0
hdci	200	false_pos	2.2	3.4	10.1	25.4
hdci	400	false_pos	1.2	2.1	6.6	19.3
hdci	1000	false_pos	0.4	1.1	4.9	12.8
uncorr	100	false_pos	7.8	13.1	25.5	51.1
uncorr	200	false_pos	7.9	13.1	26.1	50.9
uncorr	400	false_pos	7.8	13.3	25.2	51.3
uncorr	1000	false_pos	7.8	12.6	27.9	50.5

Figure

```
longA |>
  filter(k %in% c(0, 1)) |>
  pivot_longer(c(hdci, fwer, bh, uncorr), names_to = "rule", values_to = "flag") |>
  mutate(panel = ifelse(k == 0, "Type I error (per item, true model)",
    "Detection (misfitting item)")) |>
  filter(k == 0 | is_misfit) |>
  group_by(rule, B, n, panel) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  ggplot(aes(factor(B), v, colour = rule, group = rule)) +
  geom_line() + geom_point() +
  facet_grid(panel ~ n, scales = "free_y", labeller = label_both) +
  labs(x = "Bootstrap iterations (B)", y = "Percent", colour = "Rule") +
  theme_minimal(base_size = 11)
```



Study B: Monte-Carlo stability of the decision

Error rates averaged over datasets can look fine while individual decisions remain unstable. Here the *dataset is held fixed* and only the bootstrap seed varies, isolating Monte-Carlo noise from sampling noise. If an analyst re-runs `RMitemInfitCutoff()` on the same data with a different seed, how often does the flag change?

```
RES_B <- "infit_fwer_itations_B.rds"
N_SEEDS <- 40

fixed_sets <- list()
for (nn in c(250, 500)) {
  set.seed(4242 + nn)
  fixed_sets[[as.character(nn)]] <- masters[["3"]]$data[sample(10000, nn), ]
}

gridB <- expand.grid(seed = 1:N_SEEDS, B = BS, n = c(250, 500))

one_rep_B <- function(seed, B, n) {
  d <- fixed_sets[[as.character(n)]]
  out <- data.frame(item = 1:20, padj_fwer = NA_real_, p_marg = NA_real_,
                    hdci = NA_integer_)
  try({
    co <- RMitemInfitCutoff(d, iterations = B, parallel = FALSE,
                           seed = seed, verbose = FALSE)
    a <- RMitemInfit(d, cutoff = co, output = "dataframe")
    out$hdci <- as.integer(a$Flagged[match(names(d), a$item)] != "")
    f <- suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
                  alpha = ALPHA, output = "dataframe"))
    i <- match(names(d), f$item)
    out$padj_fwer <- f$padj_infit[i]
    out$p_marg <- f$p_infit[i]
  }, silent = TRUE)
  out
}

if (file.exists(RES_B)) {
```

```

simB <- readRDS(RES_B)
} else {
  resB <- mclapply(seq_len(nrow(gridB)),
    function(i) one_rep_B(gridB$seed[i], gridB$B[i], gridB$n[i]),
      mc.cores = NCORES)
  simB <- list(res = resB, grid = gridB, misfit = misfit_all)
  saveRDS(simB, RES_B)
}

longB <- do.call(rbind, lapply(seq_len(nrow(simB$grid)),
  function(i) cbind(simB$grid[i, ], simB$res[[i]]))) |>
  mutate(is_misfit = item %in% simB$misfit)

```

Decision instability

Percentage of items whose FWER flag is **not unanimous** across the 40 bootstrap seeds — i.e. the same data and the same rule give different answers depending only on the seed.

```

longB |>
  group_by(n, B, item) |>
  summarise(rate = mean(padj_fwer < ALPHA, na.rm = TRUE), .groups = "drop") |>
  group_by(n, B) |>
  summarise(unstable_items = 100 * mean(rate > 0 & rate < 1), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = unstable_items, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1)))

```

B	n=250	n=500
100	5	15
200	5	10
400	5	5
1000	0	0

Same, for the HDCl cutoff rule, as a reference point:

```

longB |>
  group_by(n, B, item) |>
  summarise(rate = mean(hdci, na.rm = TRUE), .groups = "drop") |>
  group_by(n, B) |>
  summarise(unstable_items = 100 * mean(rate > 0 & rate < 1), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = unstable_items, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1)))

```

B	n=250	n=500
100	25	35
200	15	30
400	15	25
1000	5	10

Spread of the adjusted p-value across seeds

Median across items of the within-item SD of `padj_infit` over the 40 seeds.

```

longB |>
  group_by(n, B, item) |>
  summarise(sd_padj = sd(padj_fwer, na.rm = TRUE), .groups = "drop") |>
  group_by(n, B) |>

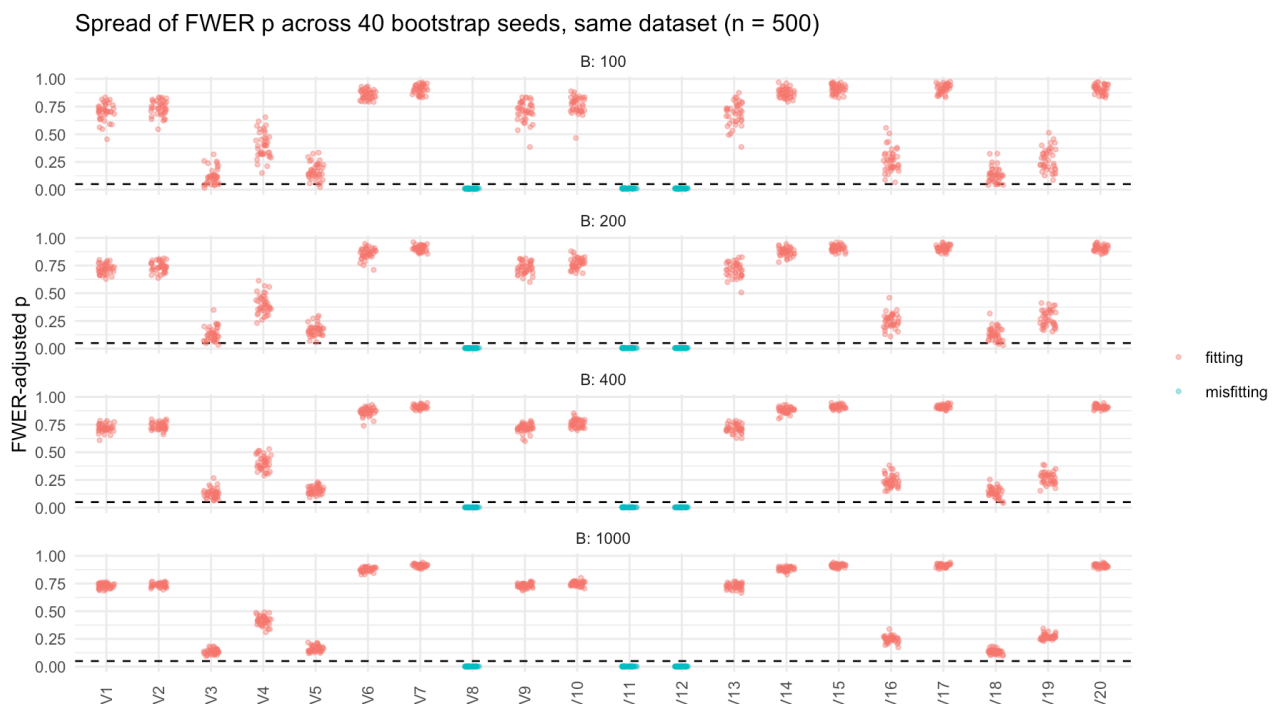
```



```
summarise(median_sd = round(median(sd_padj, na.rm = TRUE), 4), .groups = "drop") |>
pivot_wider(names_from = n, values_from = median_sd, names_prefix = "n=")
```

B	n=250	n=500
100	0.0073	0.0680
200	0.0059	0.0462
400	0.0034	0.0345
1000	0.0027	0.0191

```
longB |>
  filter(n == 500) |>
  mutate(item_lab = factor(paste0("V", item), levels = paste0("V", 1:20)),
         type = ifelse(is_misfit, "misfitting", "fitting")) |>
  ggplot(aes(item_lab, padj_fwer, colour = type)) +
  geom_hline(yintercept = ALPHA, linetype = 2) +
  geom_jitter(width = .15, alpha = .4, size = .8) +
  facet_wrap(~B, labeller = label_both, ncol = 1) +
  labs(x = NULL, y = "FWER-adjusted p", colour = NULL,
       title = "Spread of FWER p across 40 bootstrap seeds, same dataset (n = 500)") +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 90, vjust = .5))
```



Study C: the small-sample trade, and buying detection honestly

Johansson (2025) recommended 100 iterations for $n \leq 250$ because detection was highest there. Study A shows that setting carries a 23-38 percent chance of at least one false flag. But the detection loss at $B = 1000$ is real, not an illusion, so “just use 1000” is only defensible if the loss is either small or recoverable.

The key observation is that **B and `hdc_i_width` are confounded**. At $B = 100$ the .0005 and .9995 quantiles cannot be estimated, so the realised interval is narrower than the nominal 99.9 percent and the effective per-item alpha is unknown — it drifts with B. If wider flagging is wanted, the principled way to get it is to *choose* a wider alpha explicitly at large B, which is reproducible and has a stated error rate.

This study tests that: one $B = 1000$ bootstrap per replication, with the HDCl recomputed at several widths from the stored draws (`$results`), against the paper’s $B = 100 / \text{width}$.999 setting. Direction of each flag is recorded, because a false “overfit” flag and a false “underfit” flag have very different consequences in an iterative item-removal workflow.

```

RES_C <- "infit_fwer_iterations_C.rds"
WIDTHS <- c(0.999, 0.99, 0.95, 0.90)
R_C <- 200

flag_dir <- function(msq, lo, hi) {
  ifelse(msq > hi, "underfit", ifelse(msq < lo, "overfit", ""))
}

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

one_rep_C <- function(rep, k, n) {
  set.seed(3e7 + 1e6 * k + 1e3 * n + rep)
  md <- masters[[as.character(k)]]$data
  d <- md[sample(nrow(md), n), ]
  items <- names(d)
  obs <- RMitemInfit(d, output = "dataframe")
  msq <- obs$Infit_MSQ[match(items, obs$Item)]
  out <- data.frame(item = 1:20, rule = NA_character_, dir = NA_character_)[0, ]
  try({
    # paper's setting: B = 100, width .999
    c100 <- RMitemInfitCutoff(d, iterations = 100, parallel = FALSE,
                             seed = rep, verbose = FALSE)
    b <- bounds_at(c100$results, 0.999, items)
    out <- rbind(out, data.frame(item = 1:20, rule = "B=100 w=.999",
                                dir = flag_dir(msq, b[1, ], b[2, ])))
    # one B = 1000 bootstrap, several widths
    c1000 <- RMitemInfitCutoff(d, iterations = 1000, parallel = FALSE,
                              seed = rep, verbose = FALSE)
    for (w in WIDTHS) {
      b <- bounds_at(c1000$results, w, items)
      out <- rbind(out, data.frame(item = 1:20,
                                   rule = paste0("B=1000 w=", sub("^0", "", w)),
                                   dir = flag_dir(msq, b[1, ], b[2, ])))
    }
  }, silent = TRUE)
  out
}

gridC <- expand.grid(rep = 1:R_C, n = c(150, 250), k = c(0, 3))

if (file.exists(RES_C)) {
  simC <- readRDS(RES_C)
} else {
  resC <- mclapply(seq_len(nrow(gridC)),
                   function(i) one_rep_C(gridC$rep[i], gridC$k[i], gridC$n[i]),
                   mc.cores = NCORES)
  simC <- list(res = resC, grid = gridC, misfit = misfit_all)
  saveRDS(simC, RES_C)
}

longC <- do.call(rbind, lapply(seq_len(nrow(simC$grid)), function(i) {
  r <- simC$res[[i]]
  if (nrow(r) == 0) return(NULL)
  cbind(simC$grid[i, ], r)
})) |>
mutate(is_misfit = mapply(function(k, it) it %in% simC$misfit[seq_len(k)],
                           k, item),
       flag = as.integer(dir != ""))

```

Detection and false positives at n = 150 and 250 (k = 3)

```
longC |>
  filter(k == 3) |>
  mutate(type = ifelse(is_misfit, "detection", "false_pos")) |>
  group_by(rule, n, type) |>
  summarise(v = 100 * mean(flag), .groups = "drop") |>
  pivot_wider(names_from = c(type, n), values_from = v) |>
  mutate(across(where(is.numeric), \(x) round(x, 1)))
```

rule	detection_150	false_pos_150	detection_250	false_pos_250
B=100 w=.999	77.5	4.8	92.8	7.2
B=1000 w=.9	94.5	18.3	99.0	23.6
B=1000 w=.95	90.2	10.4	98.2	15.4
B=1000 w=.99	78.5	2.9	93.2	5.1
B=1000 w=.999	64.3	0.6	86.5	1.4

Family-wise false positive rate under a true model (k = 0)

```
longC |>
  filter(k == 0) |>
  group_by(rule, n, rep) |>
  summarise(any = as.integer(sum(flag) > 0), .groups = "drop") |>
  group_by(rule, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1)))
```

rule	n=150	n=250
B=100 w=.999	30.0	26
B=1000 w=.9	87.0	87
B=1000 w=.95	65.5	60
B=1000 w=.99	20.0	14
B=1000 w=.999	5.5	2

Direction of the false flags (k = 3, fitting items only)

```
dir_split <- function(kk) {
  longC |>
    filter(k == kk, if (kk == 3) !is_misfit else TRUE, dir != "") |>
    group_by(rule, sample_n = n, dir) |>
    summarise(ct = n(), .groups = "drop") |>
    group_by(rule, sample_n) |>
    mutate(pct = round(100 * ct / sum(ct), 1)) |>
    ungroup() |>
    select(rule, sample_n, dir, pct) |>
    pivot_wider(names_from = dir, values_from = pct, values_fill = 0)
}
dir_split(3)
```

rule	sample_n	overfit	underfit
B=100 w=.999	150	97.5	2.5
B=100 w=.999	250	100.0	0.0
B=1000 w=.9	150	93.7	6.3

rule	sample_n	overfit	underfit
B=1000 w=.9	250	98.8	1.2
B=1000 w=.95	150	94.9	5.1
B=1000 w=.95	250	99.0	1.0
B=1000 w=.99	150	99.0	1.0
B=1000 w=.99	250	100.0	0.0
B=1000 w=.999	150	100.0	0.0
B=1000 w=.999	250	100.0	0.0

The same split under a **true model** ($k = 0$), where every flag is a false positive:

```
dir_split(0)
```

rule	sample_n	overfit	underfit
B=100 w=.999	150	51.5	48.5
B=100 w=.999	250	64.6	35.4
B=1000 w=.9	150	46.5	53.5
B=1000 w=.9	250	48.7	51.3
B=1000 w=.95	150	40.4	59.6
B=1000 w=.95	250	52.6	47.4
B=1000 w=.99	150	48.8	51.2
B=1000 w=.99	250	46.7	53.3
B=1000 w=.999	150	45.5	54.5
B=1000 w=.999	250	25.0	75.0

Expected flags per analysis ($k = 3$)

The quantity an analyst actually faces: how many of the 3 misfitting items are flagged, and how many of the 17 fitting items are flagged, in a single run.

```
longC |>
  filter(k == 3) |>
  group_by(rule, n, rep) |>
  summarise(true_flags = sum(flag[is_misfit]),
            false_flags = sum(flag[!is_misfit]), .groups = "drop") |>
  group_by(rule, n) |>
  summarise(true = round(mean(true_flags), 2),
            false = round(mean(false_flags), 2), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = c(true, false))
```

rule	true_150	true_250	false_150	false_250
B=100 w=.999	2.33	2.79	0.81	1.23
B=1000 w=.9	2.84	2.97	3.10	4.01
B=1000 w=.95	2.70	2.94	1.76	2.61
B=1000 w=.99	2.36	2.80	0.50	0.86
B=1000 w=.999	1.93	2.60	0.10	0.23

Conclusions

1. Marginal p-values are calibrated at every B — including 100. Per-item false positives under a true model were 4.25-5.15 percent at B = 100 and 4.25-4.70 at B = 1000, with no trend. The documentation claim that marginal p-values are “well calibrated even at a few hundred iterations” is confirmed, and can be stated more strongly: 100 iterations is enough for calibration.

2. FWER is close to B-invariant in both error rate and power. Detection of a single on-target misfitting item was 99-100 percent at every B from 100 to 1000. The family-wise error rate fell only mildly, from 9/3/2/10 percent (n = 150/250/500/1000) at B = 100 to 2/2/3/6 at B = 1000. The documented “liberal when the simulation is small” behaviour is real but small — a few percentage points, partly within Monte-Carlo noise at 100 replications. It is not the main reason to run more iterations.

3. The real cost of small B is decision instability, which averaged error rates hide. Holding the dataset fixed and varying only the bootstrap seed, the percentage of items whose FWER flag was *not unanimous* across 40 seeds was 15 percent at B = 100 and 0 percent at B = 1000 (n = 500). The median within-item SD of the adjusted p-value fell from .068 to .019. Critically, the three genuinely misfitting items were flagged in 100 percent of seeds at every B — **all of the instability was in fitting items flipping in and out of significance**. Small B therefore costs reproducibility and produces spurious flags, not missed detections. This is the argument for ≥ 1000 iterations, and it is a different argument from the one currently in the documentation.

4. BH has a hard reachability floor that FWER does not. The smallest attainable BH-adjusted p-value is $m / (B + 1)$ for m comparisons, so BH cannot flag a single comparison unless $B \geq m/\alpha - 1$. With m = 20 items at $\alpha = .05$ that is $B \geq 399$. This is arithmetic, not power, and it is visible in the results: BH detection of one misfitting item was 1 / 8 / 99 / 100 percent at B = 100 / 200 / 400 / 1000. The jump at B = 400 is exactly where $20/401 = .0499$ crosses below .05. FWER has no such floor — the studentised-max p-value needs only $B \geq 1/\alpha - 1 = 19$.

5. HDCl cutoffs are strongly B-dependent, and B = 100 is badly calibrated. Under a true model the probability of at least one spuriously flagged item was 38 / 23 / 33 / 28 percent at B = 100, falling to 2 / 3 / 3 / 5 percent at B = 1000. Per-item this is 2.30 percent versus 0.10 percent at n = 150. This explains the Johansson (2025) Study 1 finding that detection *falls* as iterations increase (80.0 / 76.8 / 69.4 percent at n = 150): at small B the .0005 and .9995 quantiles cannot be estimated, the HDCl band is too narrow, and the rule over-flags. The apparent power advantage of 100 iterations is that artifact, not genuine sensitivity. The paper’s per-item false-positive rates at B = 100 (roughly 1.4-2.3 percent here) fall below the 5 percent threshold used for highlighting in its figures, but aggregate to a 23-38 percent chance of at least one false flag across 20 items.

6. The small-sample detection loss is real, but it is a `hdci_width` question, not an iterations question. Study C shows that B = 1000 with `hdci_width = 0.99` reproduces the detection rate of the Johansson (2025) setting (B = 100, width .999) while being better on every other measure:

Rule	det n=150	fp n=150	det n=250	fp n=250	family-wise FP (true model) n=150 / n=250
B=100 w=.999	77.5	4.8	92.8	7.2	30.0 / 26.0
B=1000 w=.99	78.5	2.9	93.2	5.1	20.0 / 14.0
B=1000 w=.999	64.3	0.6	86.5	1.4	5.5 / 2.0

Expected flags per analysis at n = 150: 2.33 true / 0.81 false for the B = 100 setting, versus 2.36 true / 0.50 false at B = 1000 with width .99. Equal detection, roughly 40 percent fewer false flags, a *stated* error rate, and a reproducible result. The two settings are not a trade-off — one dominates.

7. The “false positives are mostly overfit, so they are benign” argument does not extend to the null case. With three misfitting items present, 97-100 percent of false flags were *overfit*, reproducing Table 2 of Johansson (2025) and consistent with the overfit cascade. But under a **true model** the split was roughly 50/50 overfit/underfit at every width. Table 2 was computed on datasets that *contained* misfitting items, so it does not license the general claim that false flags are harmless. About half the flags raised by an over-liberal rule on a well-fitting scale point the wrong way in the direction that triggers item deletion.

Practical recommendations

Use	Iterations
HDCl cutoffs (<code>RMitemInfit</code> with <code>cutoff</code>)	≥ 1000 always; tune <code>hdci_width</code> , not B
Small samples (n $\leq$ 250) wanting more sensitivity	<code>iterations = 1000</code> , <code>hdci_width = 0.99</code> — dominates <code>iterations = 100</code> , <code>hdci_width = 0.999</code>
Marginal p-value, single pre-specified item	100 is adequate; 1000 for a stable reported value
FWER (<code>correction = "fwer"</code>)	≥ 1000 , for reproducibility rather than calibration

Use	Iterations
BH (<code>correction = "fdr_bh"</code>)	$B \geq m/\alpha - 1$, i.e. ≥ 400 for 20 items — and far more for pairwise comparisons

The existing ≥ 1000 warning threshold is well chosen. The stated *reason* should change: the dominant problem at small B is Monte-Carlo instability of the decision (and, for BH, unreachability), not liberal FWER calibration.

The reason B and `hdc_i_width` must not be conflated: at $B = 100$ the .0005 and .9995 quantiles cannot be estimated — the HDCI endpoints are effectively the observed min and max of 100 draws — so the realised interval is narrower than nominal and the effective per-item alpha is unknown and drifts with B. That is visible in the family-wise rate at $B = 100$, which wanders (38 / 23 / 33 / 28 percent across n in Study A) instead of settling. Choosing a wider `hdc_i_width` at large B gives the same sensitivity with an error rate that is stated, stable across n, and reproducible across seeds.

Note on other functions using this helper

`.bootstrap_pvalues()` is shared with the Q3, partial-gamma DIF/LD, and CFA cutoff functions, so point 4 applies to them with much larger m. Verified directly for Q3: with 20 items (190 pairs) at $B = 1000$, on data with three genuinely locally dependent pairs sitting at the marginal p floor of $1/1001 = .000999$, `correction = "fdr_bh"` returns adjusted p = .0633 for all three and flags **none**, while `correction = "fwer"` on the same bootstrap flags all three. A single pair would need $B \geq 190/.05 - 1 = 3799$. The package default is "fwer", so the default path is unaffected, but the BH option is unusable at the documented iteration counts for pairwise comparisons.

References

- Ferreira, J. A. (2024). The Benjamini-Hochberg method with infinitely many discoveries. *Journal of Statistical Planning and Inference*.
- Johansson, M. (2025). Detecting Item Misfit in Rasch Models. *Educational Methods & Psychometrics*, 3(18). <https://dx.doi.org/10.61186/emp.2025.5>
- Westfall, P. H., & Young, S. S. (1993). *Resampling-Based Multiple Testing*. Wiley.